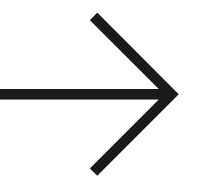
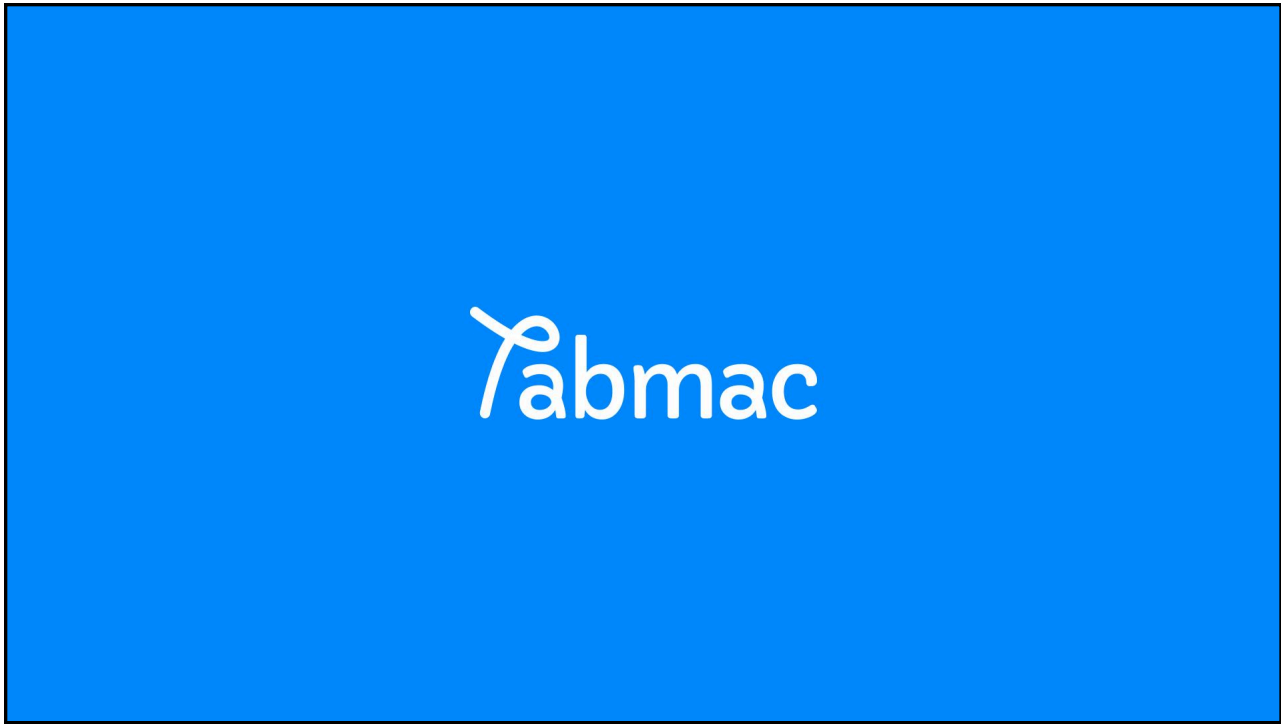


CHRIS JIA

product designer & builder

PORTFOLIO/作品集





Tabmac

餐饮/移动端 & Web/2025 - 2026

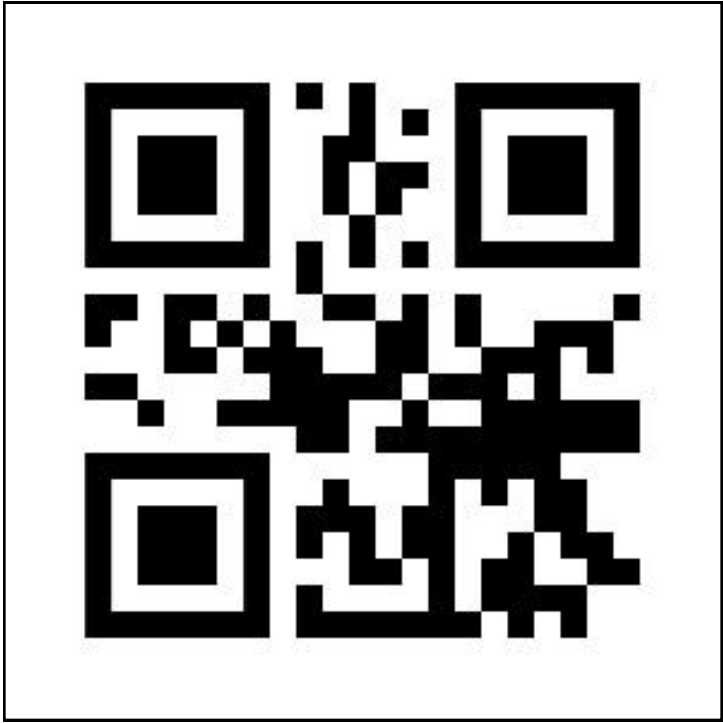
总览	3
了解更多	4



Treeing

旅行/移动端 & Web/2023 - 2025

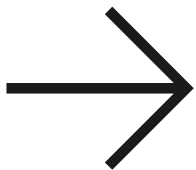
总览	9
了解更多	10



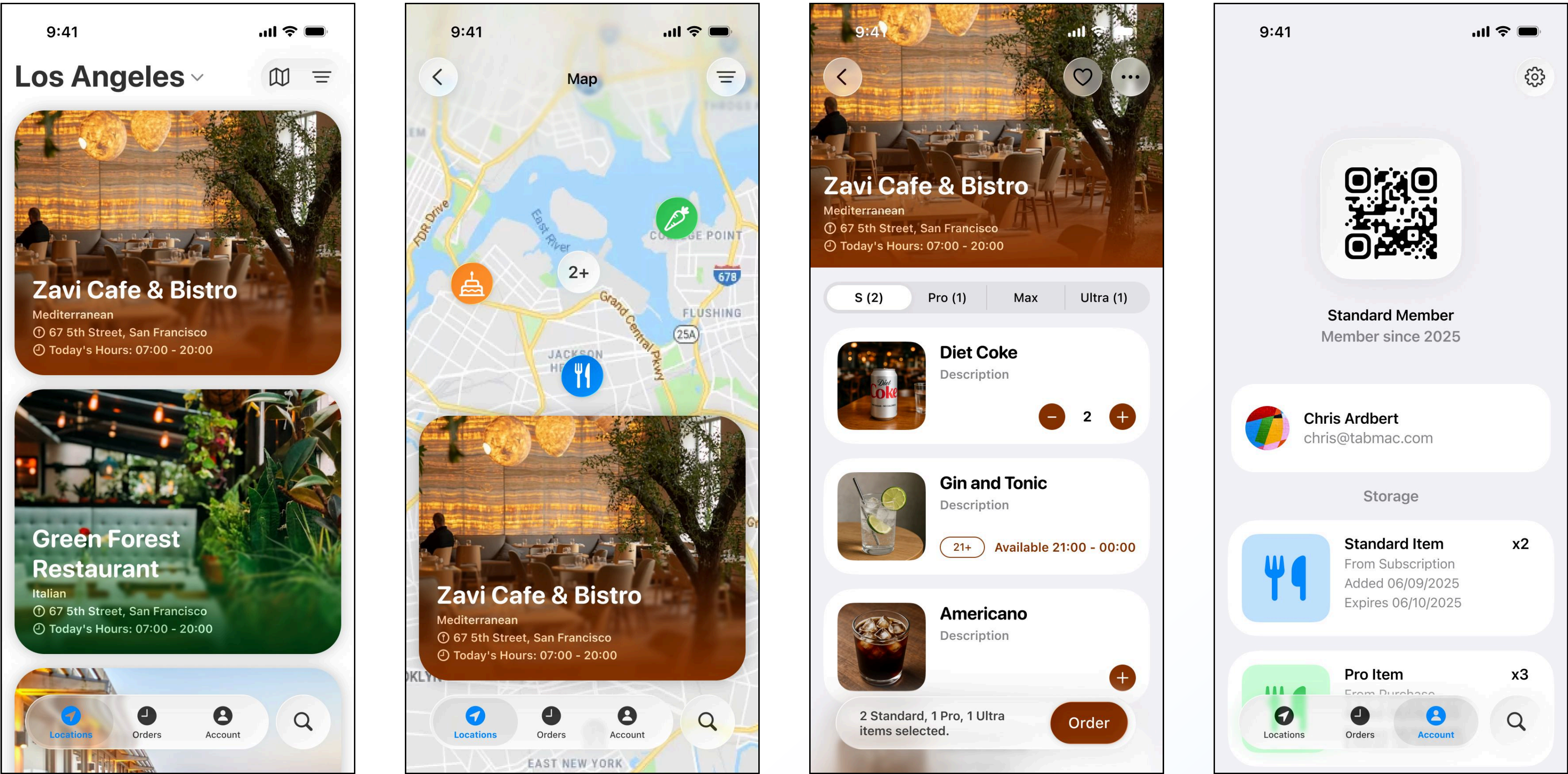
更多项目

访问可交互式在线作品集以浏览更多项目信息。

<https://jiia.cc>

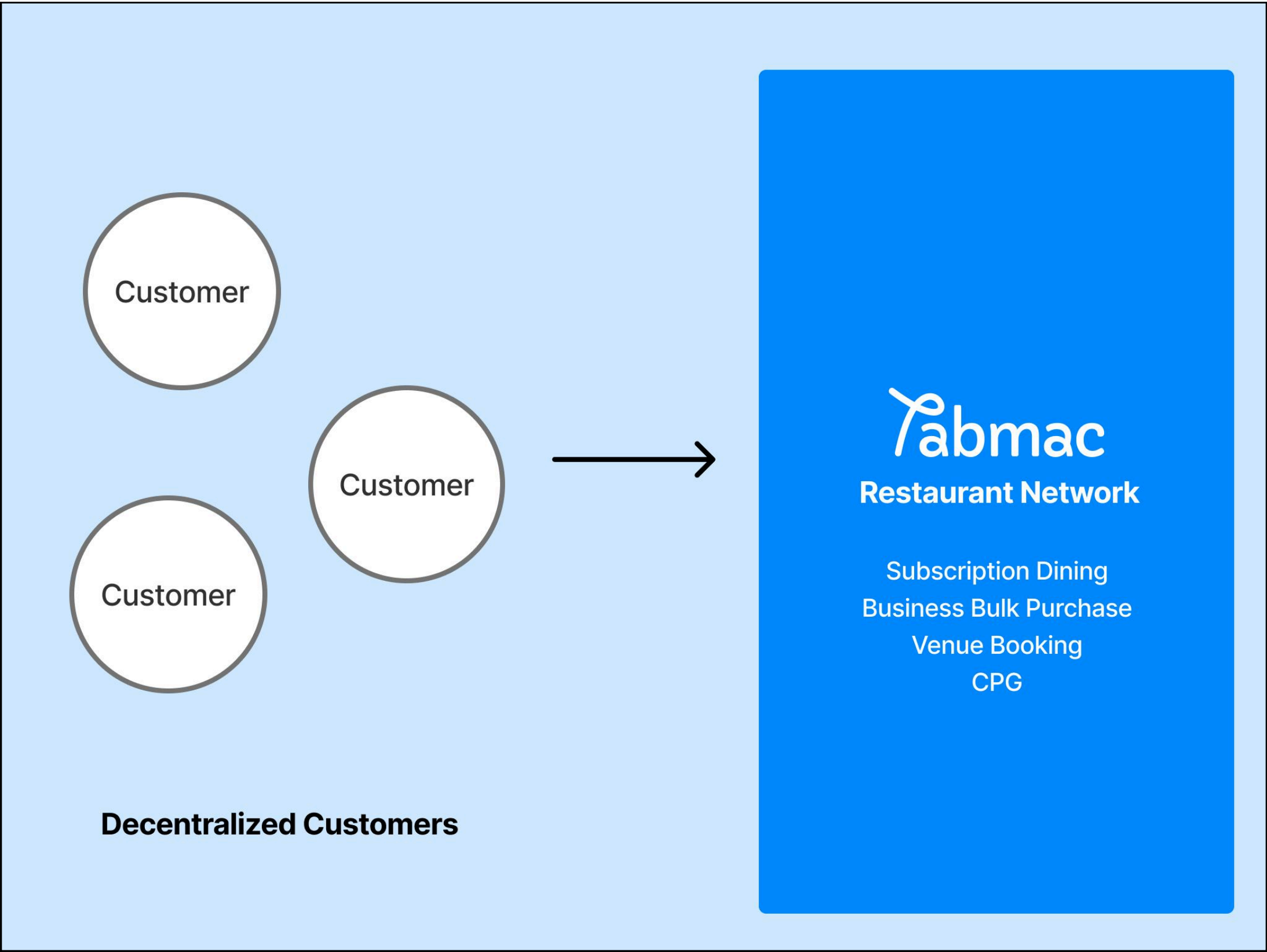


Tabmac 是一个订阅制餐饮平台，连接用户与精选餐厅体验，同时为企业采购、场地预订和结账流程提供一体化解决方案。



Tabmac: 为什么餐饮订阅不只是会员权益

从一次性用餐决策转向可持续使用的餐饮额度网络。



餐饮行业本身并不是天然订阅型消费。大多数用餐决策发生在一次次独立场景里，用户每次都要重新发现和比较，餐厅也很难沉淀成可预测的复购关系。

Tabmac 的机会不是给餐厅加一层普通会员权益，而是把分散的餐饮消费重新组织成订阅模型。通过不同档位和餐饮额度，用户购买的不再是单次折扣，而是在合作餐厅网络中持续使用、自由组合的用餐能力。

因此 Tabmac 的核心不是卖套餐，而是建立一个订阅制餐饮网络。

Tabmac: 把消费者、企业和餐厅放进同一条价值链

同一套餐饮额度网络，让三类角色进入同一条增长逻辑。

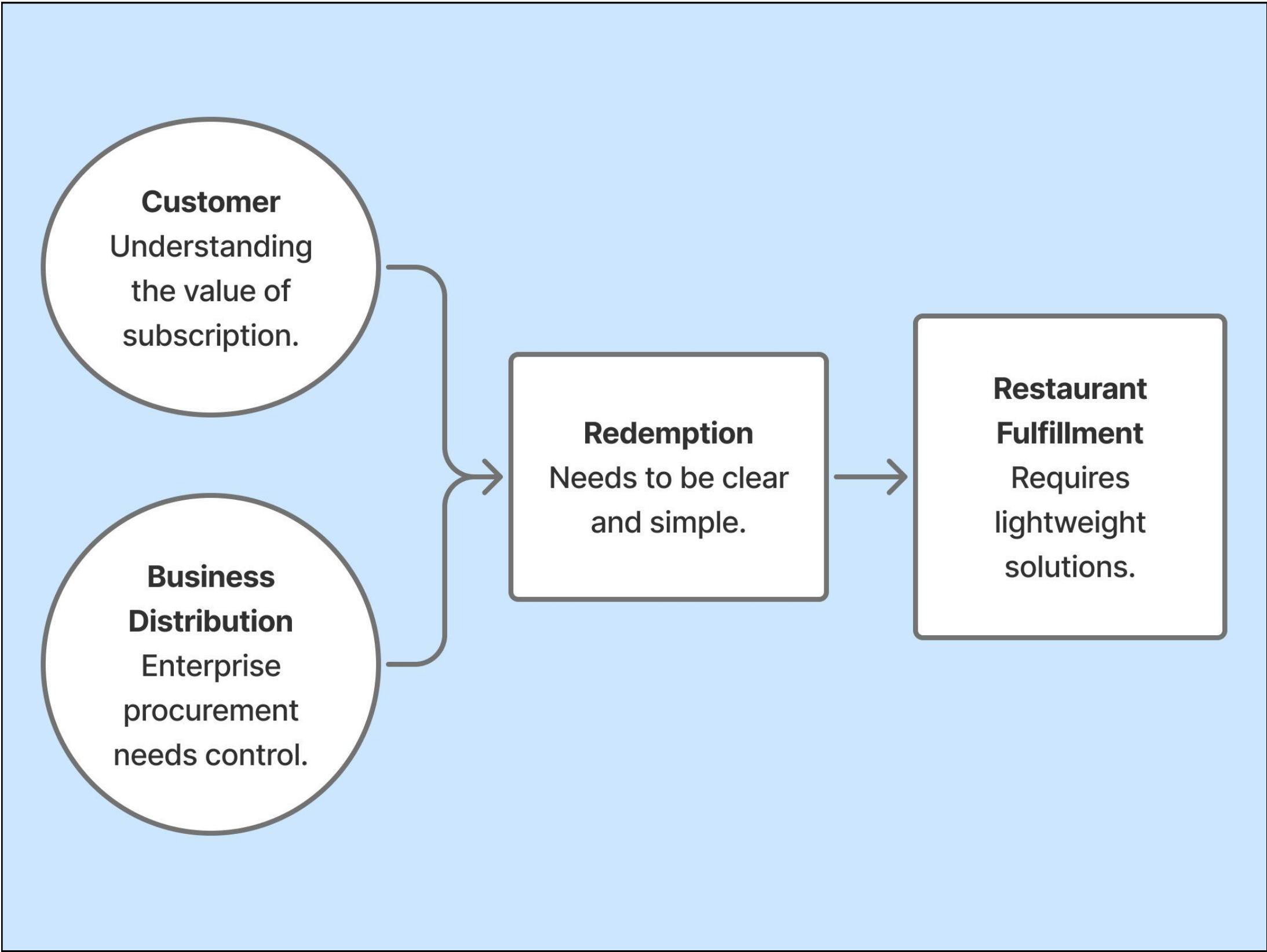
Tabmac 的商业逻辑不是把消费者、企业和餐厅分别服务好，而是让它们通过同一套餐饮额度网络进入同一条价值链。

平台价值来自这些角色之间的互相增强：餐饮额度创造持续需求，更多需求吸引更多餐厅，更多餐厅又让订阅和企业采购更有价值。



Tabmac: 从价值不清到履约断裂

把用户理解、企业采购、到店兑换和餐厅履约看成一条连续链路。



在梳理 Tabmac 的使用场景时，我把问题看成一条连续链路：用户是否理解订阅价值，用户是否能低成本完成兑换，企业是否能采购和分发，餐厅是否能顺畅履约。

第一段断裂发生在价值理解上，第二段断裂发生在到店兑换上。企业侧还需要继续管理预算、成员、分发规则和使用记录；餐厅侧则不能因为加入平台而增加过重的解释和后台维护成本。

把问题拆成可设计、可验证的模块。

这些断裂点共同指向一个产品判断：订阅要成立，价值解释、组织控制、兑换路径和餐厅履约必须被同一系统承接。



订阅价值需要被解释

用户需要理解档位、餐饮额度和可选餐厅，而不只是看到优惠描述。



企业采购需要控制

预算、成员、分发规则和使用记录需要进入同一管理模型。



兑换路径不能过重

到店场景必须支持低摩擦兑换，避免安装、注册和规则理解成本。



餐厅履约需要轻量

餐厅需要稳定需求，同时减少员工解释和后台维护成本。

Tabmac: 核心假设：用订阅连接发现、兑换和履约

订阅不是购买入口，而是系统基础。

如果 Tabmac 能用档位和餐饮额度建立清晰的餐饮价值结构，并让额度在消费者使用、企业分发和餐厅履约中保持一致，那么订阅就会成为连接需求、供给和履约的系统基础。



清晰的额度模型

让用户进入流程时就理解档位、餐饮额度和可用范围。



低摩擦履约

兑换、结账和小费需要适配真实到店场景。



组织级控制

企业采购需要覆盖购买、分发、团队和使用追踪。



餐厅侧价值

餐厅需要看到订阅、企业和包场带来的复合需求。

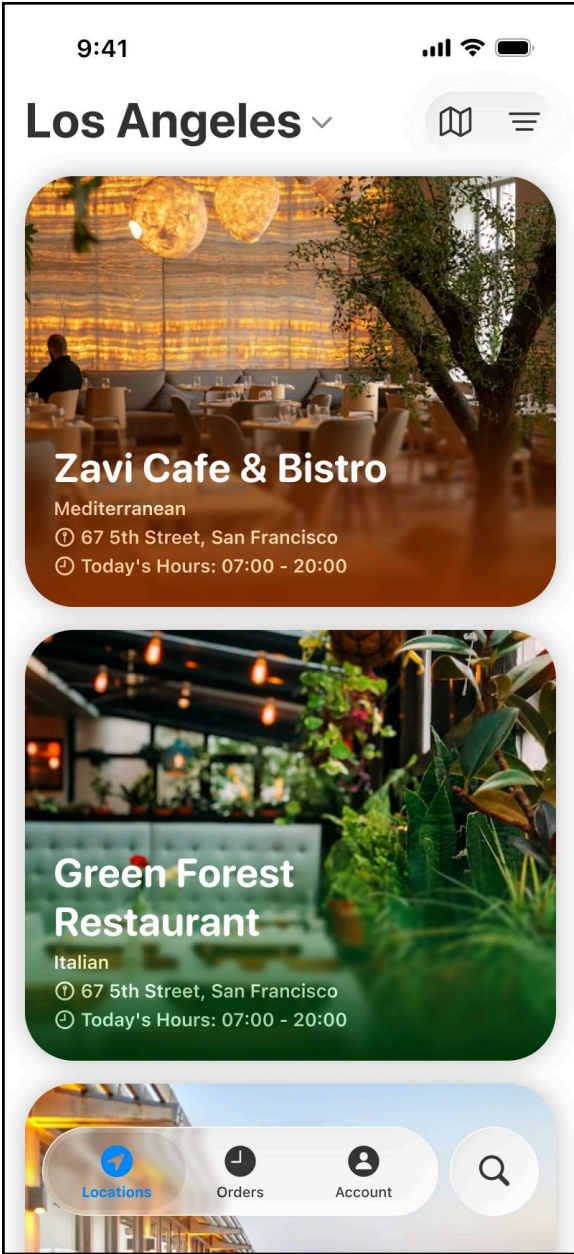


生态增长回路

每个模块都应反过来增强订阅网络和餐厅供给。

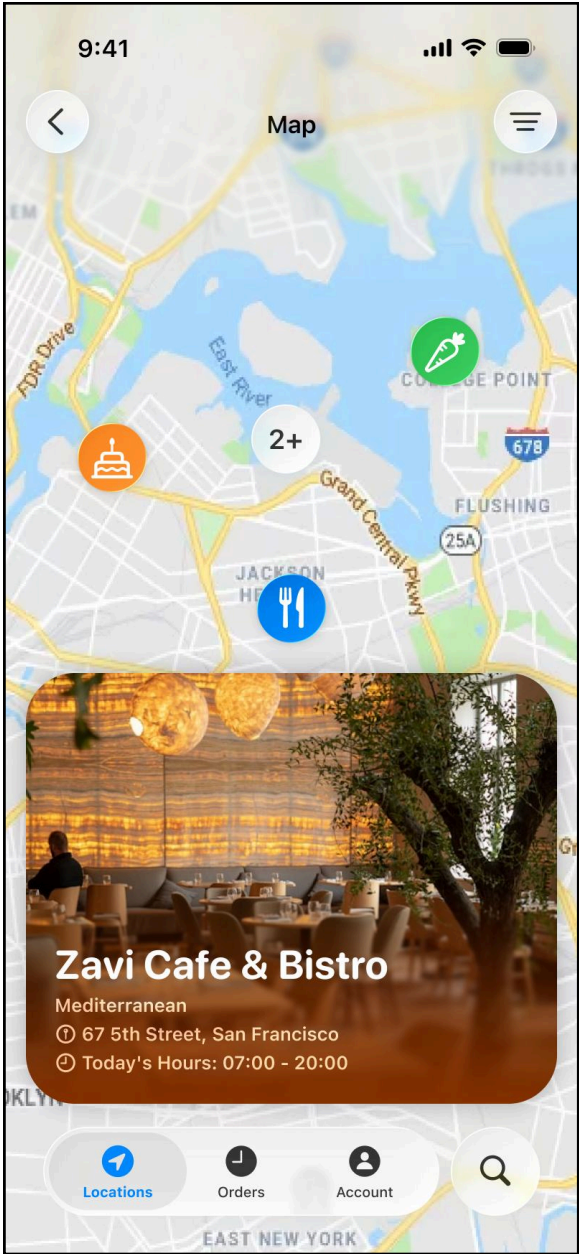
Tabmac: V1：先把消费侧闭环做清楚

先验证用户是否能理解订阅、发现餐厅、确认权益并完成兑换。



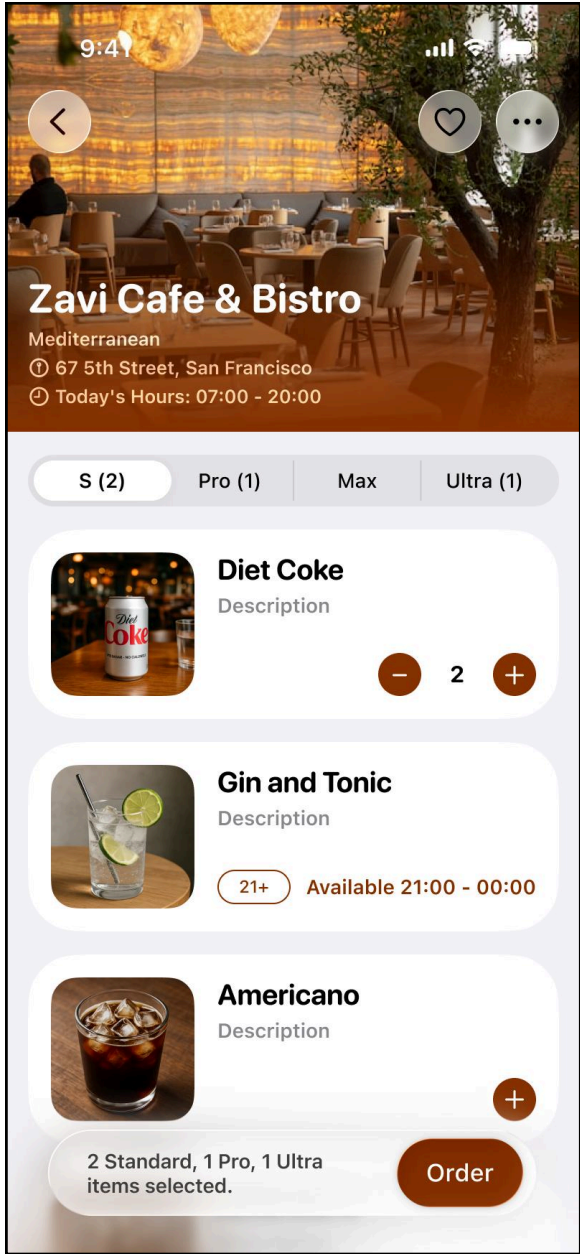
Main

让餐厅、档位和可用餐饮额度在首页直接可见。



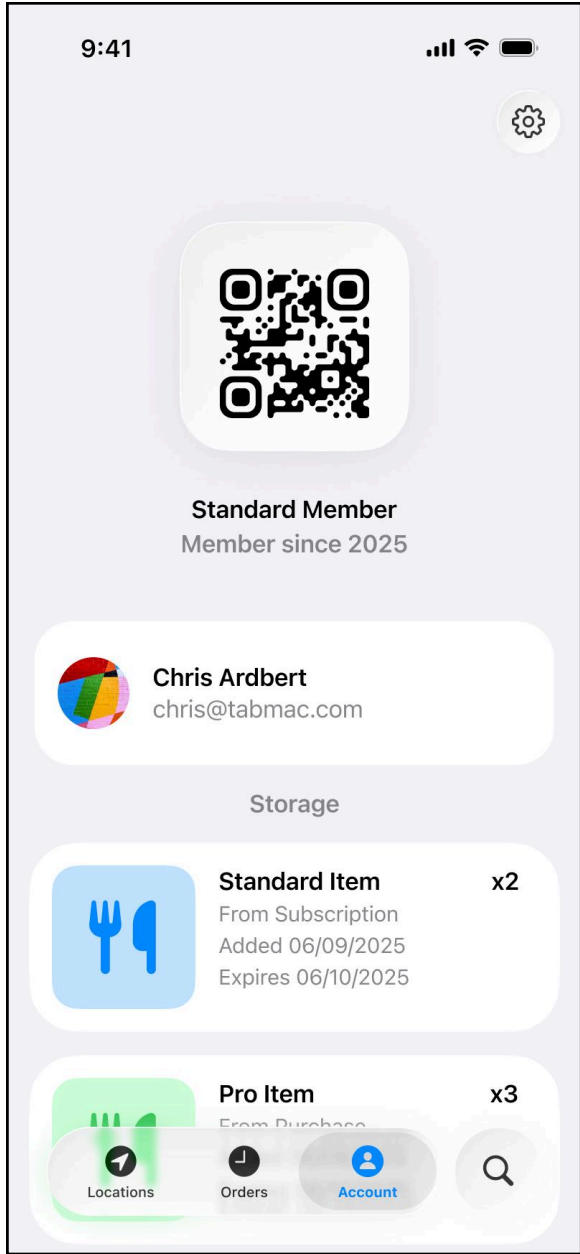
Map

用位置把订阅权益转化成真实到店选择。



Store

在门店页集中解释可用内容、规则和兑换入口。



Account

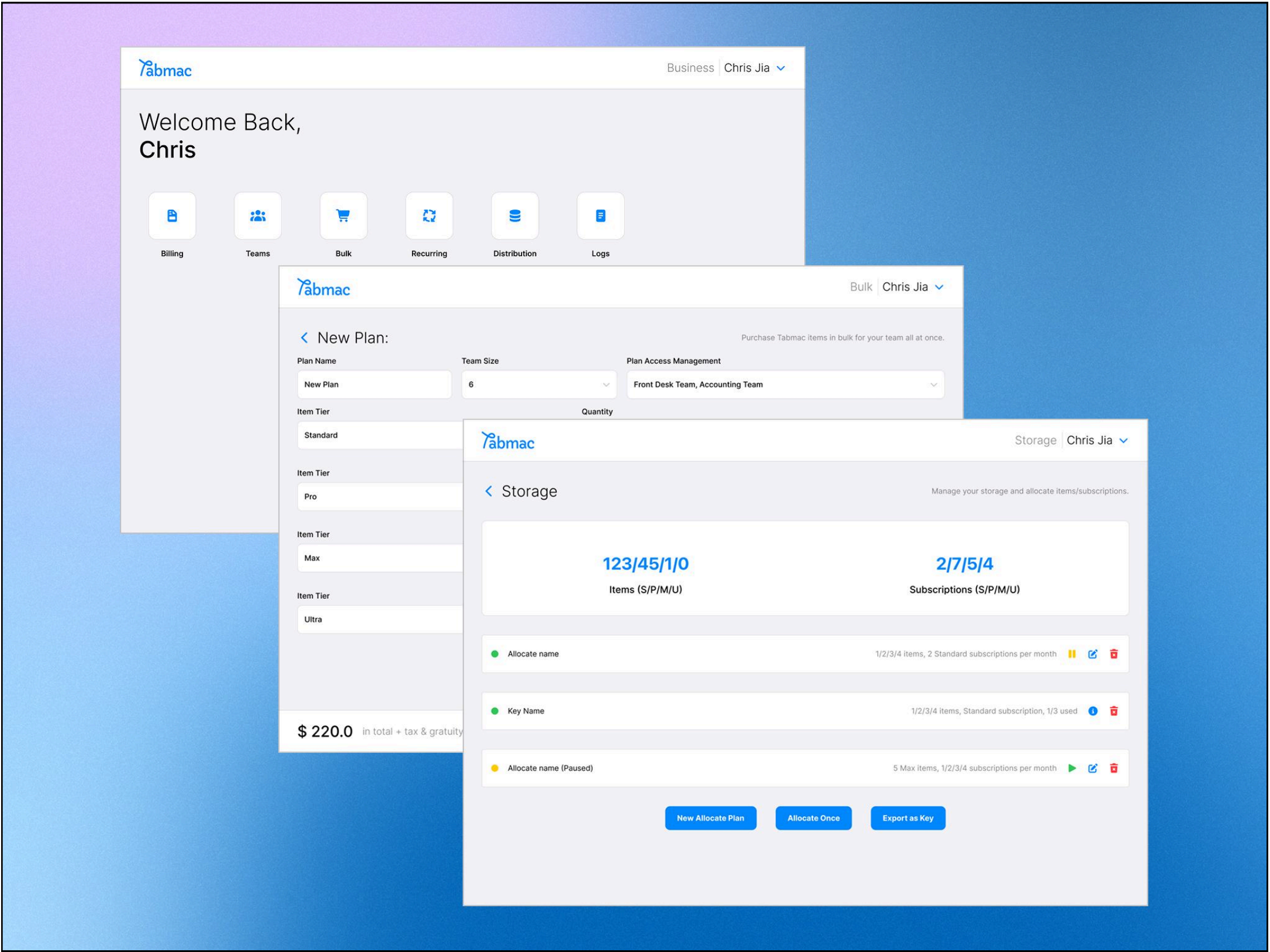
统一承接套餐、餐饮额度、奖励和企业分发权益。

Tabmac 的第一版没有从后台能力开始，而是先验证消费侧闭环。主界面把餐饮订阅变成可以浏览的具体选择，用户看到的是餐厅、档位、可用餐饮额度和行动入口。

地图帮助用户把餐饮额度转化为真实地理选择，门店页集中解释这家餐厅能用什么、规则是什么、如何兑换。账户页则统一承接套餐、餐饮额度、奖励和企业分发权益。

Tabmac: 让企业采购从一次购买变成可管理流程

把同一套餐饮额度扩展到组织采购和福利分发。



Business 不是独立的企业后台，而是把同一套餐饮额度从个人消费扩展到组织采购和福利分发。真正复杂的部分不是付款，而是购买后如何分发、如何持续购买、如何让成员使用、如何追踪结果。

我把 Bulk Purchase、Recurring Purchase、Distribution 和 Teams 拆成清晰的管理任务，而不是把所有设置压进一个购买表单。购买行为和组织管理之间保持清晰边界，也让后续追踪更容易。

企业侧不只需要购买入口，更需要后续管理结构。

管理员可以先完成批量或周期性购买，再决定分发对象和团队范围。体验上，Business 更像任务型后台而不是营销页。



Bulk Purchase

按人数、预算或权益类型批量购买餐饮额度或订阅。



Recurring Purchase

支持周期性发放，让餐饮福利持续进入组织流程。



Distribution

按成员、团队或规则分发购买结果。



Teams

用组织结构承接预算、成员和使用记录。

Tabmac: 把包场预订从来回沟通变成即时判断

用酒店预订式结构前置容量、日期、最低消费和规则。

Venue

Search

Q

Los Angeles, CA

Perfect Venue for Parties

67 5th Street, San Francisco, CA, USA

10-100

About this venue

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc at scelerisque urna, sed scelerisque nisi. Quisque hendrerit dui sit amet nisi viverra, id ullamcorper ex eleifend. Nunc mi erat, iaculis eu nisi non, consequat pellentesque mi. Vivamus tempor ante nec ullamcorper rutrum. Phasellus at ligula mauris. Donec aliquam luctus tincidunt. Sed suscipit, dui scelerisque feugiat efficitur, libero lorem feugiat diam, id gravida elit velit quis magna. Proin mattis varius sem eu semper. Nulla semper, eros non condimentum aliquet, metus odio pretium felis, sed condimentum diam tellus id sem...

Show more

3D Tour

Share

Like

From \$60 per person

Get a personalized quote based on your event needs

Instant Quote

Add to list

传统包场预订的痛点不是用户不知道要订，而是双方需要在容量、日期、最低消费、菜单、服务费、取消规则和特殊需求之间持续沟通。每一次条件变化都可能触发新的报价和确认。

Tabmac Venue 借用了酒店预订的结构：先选择场地、时间和人数，再查看容量、规则和价格，最后进入确认。Instant Quote 把原本需要多轮沟通确认的条件前置成可判断的报价结构。

减少低确定性的沟通，让请求更接近可成交状态。

Venue 的关键不是把包场表单搬到线上，而是让用户尽早判断是否合适，同时让餐厅通过规则配置保留控制权。



场地发现

先让用户理解场地、容量、约束和适用场景。



Instant Quote

把价格和规则前置，减少低确定性的来回沟通。

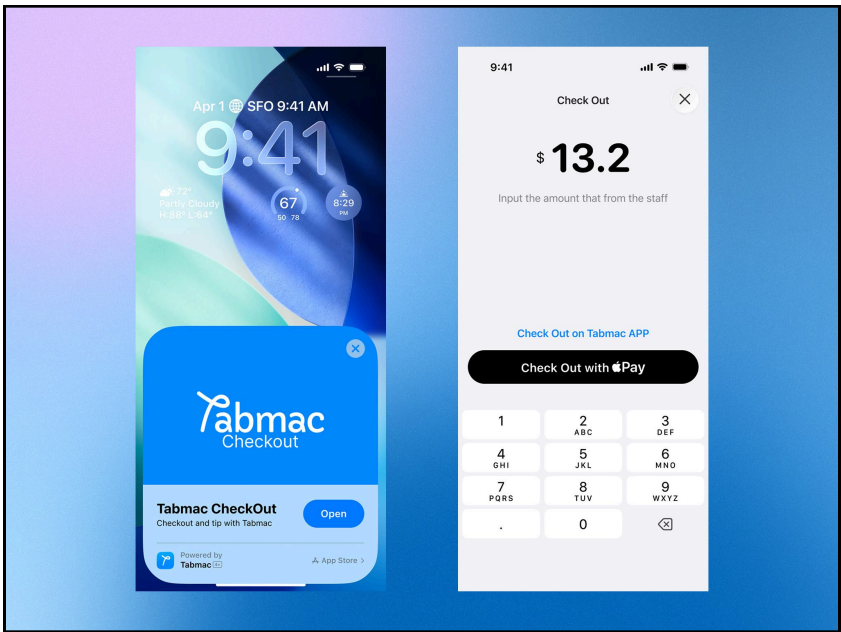


餐厅控制

餐厅通过配置规则保留对报价和可订状态的控制。

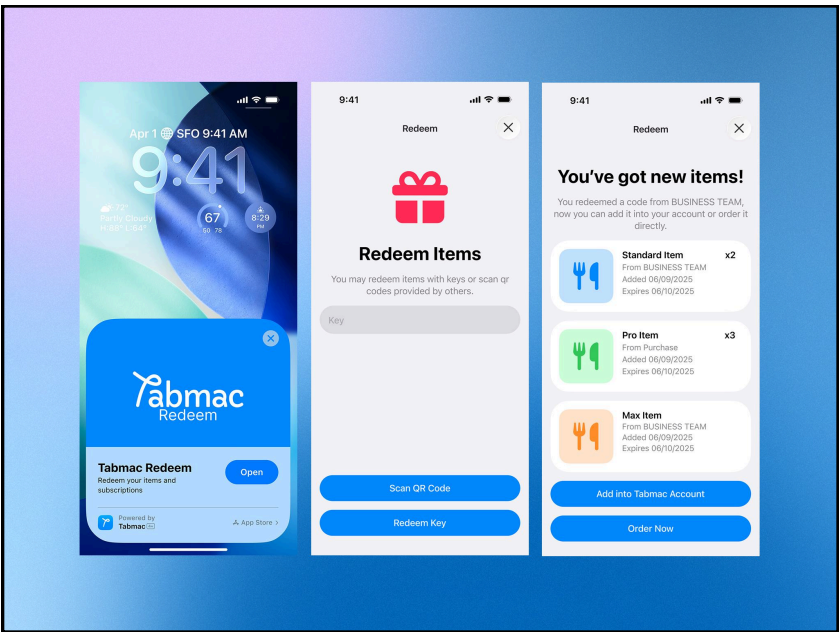
Tabmac: 把结账和兑换做成低摩擦入口

Checkout、Redeem 和 Tips 共同降低到店履约摩擦。



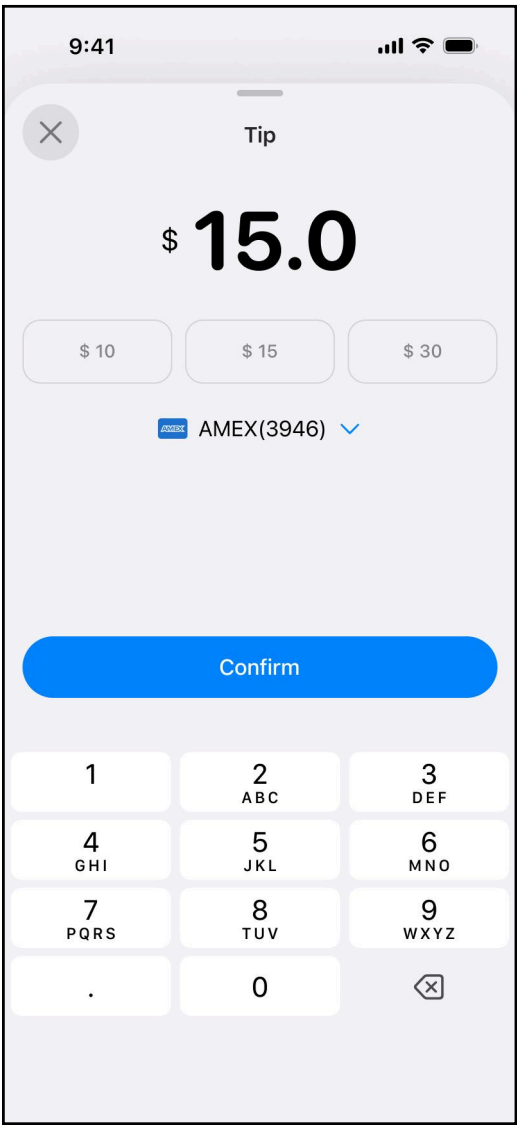
Checkout

用 App Clips 降低首次支付门槛。



Redeem

让订阅、活动和企业分发权益共用同一兑换路径。



Tips

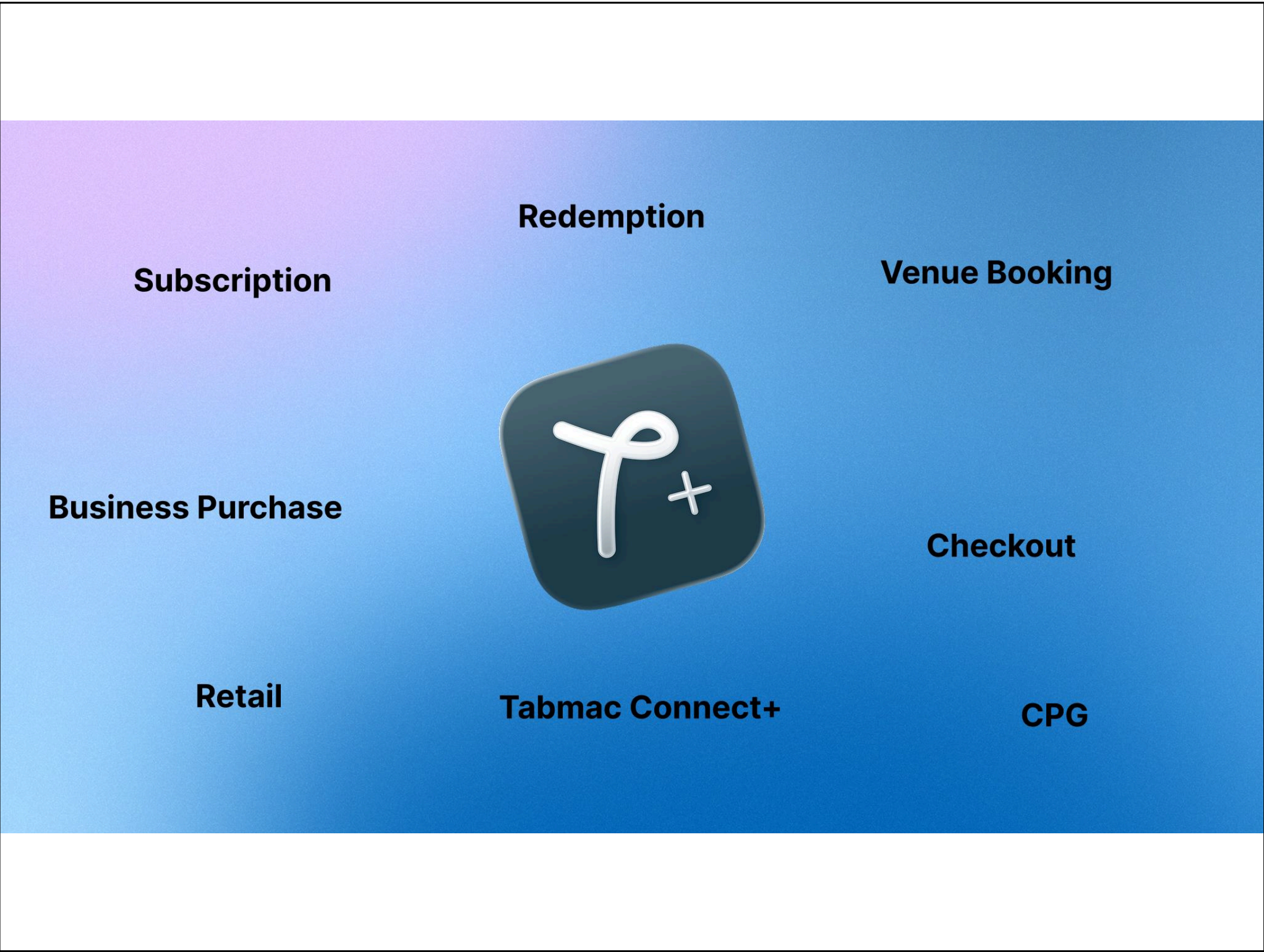
把小费、企业政策和最终支付确认整合到同一流程。

到店场景里的用户耐心很低。结账、兑换和小费如果要求用户安装完整应用、创建账户或理解复杂规则，就会直接影响完成率，也会增加餐厅员工解释成本。

Checkout 和 Redeem 都被设计成轻量入口。通过 App Clips，未安装完整应用的用户也可以完成支付。Tips 被当作支付和履约规则的一部分，与企业政策、餐厅设置和用户确认放在同一流程里。

Tabmac: 从功能集合到餐饮需求系统

订阅、Business、Venue 和履约层连接成增长结构。



Tabmac 的最终设计不是功能集合，而是一套围绕餐饮额度运转的餐饮需求系统。需求基础是订阅和餐饮额度，用户通过不同档位理解餐饮价值，并在合作餐厅网络中自由组合体验。

需求扩展来自 Business 和 Venue。企业采购把餐饮预算带入组织场景，包场预订把团队活动和商务需求带入餐厅网络。Checkout、Redeem 和 Tips 则降低到店支付、权益兑换和服务规则确认的摩擦。

最终解决的是不同角色能否围绕同一套餐饮额度、供给和履约系统完成各自目标。

Tabmac 的设计收益不是单个页面效率，而是让消费者、企业、餐厅和平台围绕同一套系统互相增强。



消费者

更低成本理解和使用多样餐饮选择。



企业

把餐饮福利纳入可购买、可分发、可追踪的组织流程。



餐厅

通过一个入口承接订阅、企业、包场和到店履约需求。



平台

让餐饮额度、供给、企业预算和履约模块互相推动增长。

项目反思：订阅不是定价包装，而是价值如何被理解、交付和增长。

Tabmac 让我重新理解了订阅制产品的设计重点。订阅不是把价格打包成一个计划，也不是在页面上增加更多会员权益。真正困难的地方是用档位和配额建立足够清楚的价值结构，让用户相信自己有自由度，让企业相信购买过程可控，让餐厅相信加入生态能带来持续增长。

如果只从消费侧看，问题会被理解成餐厅发现和兑换体验；如果只从企业侧看，问题会变成采购和分发工具；如果只从餐厅侧看，问题又会变成后台和支付效率。



Treeing 是一个围绕熟客关系设计的旅行平台。Host 通过 Guest List 管理常客，用户也能快速找到熟悉的 Host，并以更好的价格完成预订。

What's your plan?

Any date

Any guests

Q

≡

Recommend

My Hosts

Spotlight

Map

Filters

Secluded Cabin in the Woods
Big Bear Lake, CA, USA
\$ 185 +
152
85 km
SUPER HOST

Charming Getaway by the Coast
Monterey, CA, USA
\$ 210 +
203
120 km

Luxury Villa with Ocean View
Malibu, CA, USA
\$ 350 +
188
65 km

Cozy Apartment in Downtown
San Francisco, CA, USA
\$ 275 +
95
25 km

Modern Loft in Arts District
Los Angeles, CA, USA
\$ 220 +
125
40 km

Rustic Farmhouse with Vineyard
Napa Valley, CA, USA
\$ 410 +
168
95 km

Stylish Condo with City View
San Diego, CA, USA
\$ 290 +
110
30 km

Seaside Cottage with Private Beach
Santa Barbara, CA, USA
\$ 380 +
230
150 km

Mountain Chalet with Ski Access
Lake Tahoe, CA, USA
\$ 380 +
142
110 km

Historic Home in Old Town
Sacramento, CA, USA
\$ 250 +
105
50 km

Desert Oasis with Pool & Spa
Palm Springs, CA, USA
\$ 280 +
175
130 km

Beachfront Bungalow with Sunset View
Santa Cruz, CA, USA
\$ 220 +
215
75 km

What's your plan?

Any date

Any guests

Q

≡

← Lovely Warm Cottage

\$ 123 /night
Start from Jan 05 · Instant Confirm

Reserve

117

♥

Lake Arrowhead, CA, USA 123KM

3 Rooms

4 Beds

2 Bathrooms

Entire Place

Long-Term

Hosted by Chris

About This Listing

Discover the charm of the Lovely Warm Cottage, a cozy retreat nestled in a peaceful village. This inviting home features a sunlit living room, a fully equipped kitchen, and a snug bedroom perfect for relaxing after a day of exploring. Enjoy the serene garden and nearby walking trails for a refreshing escape.

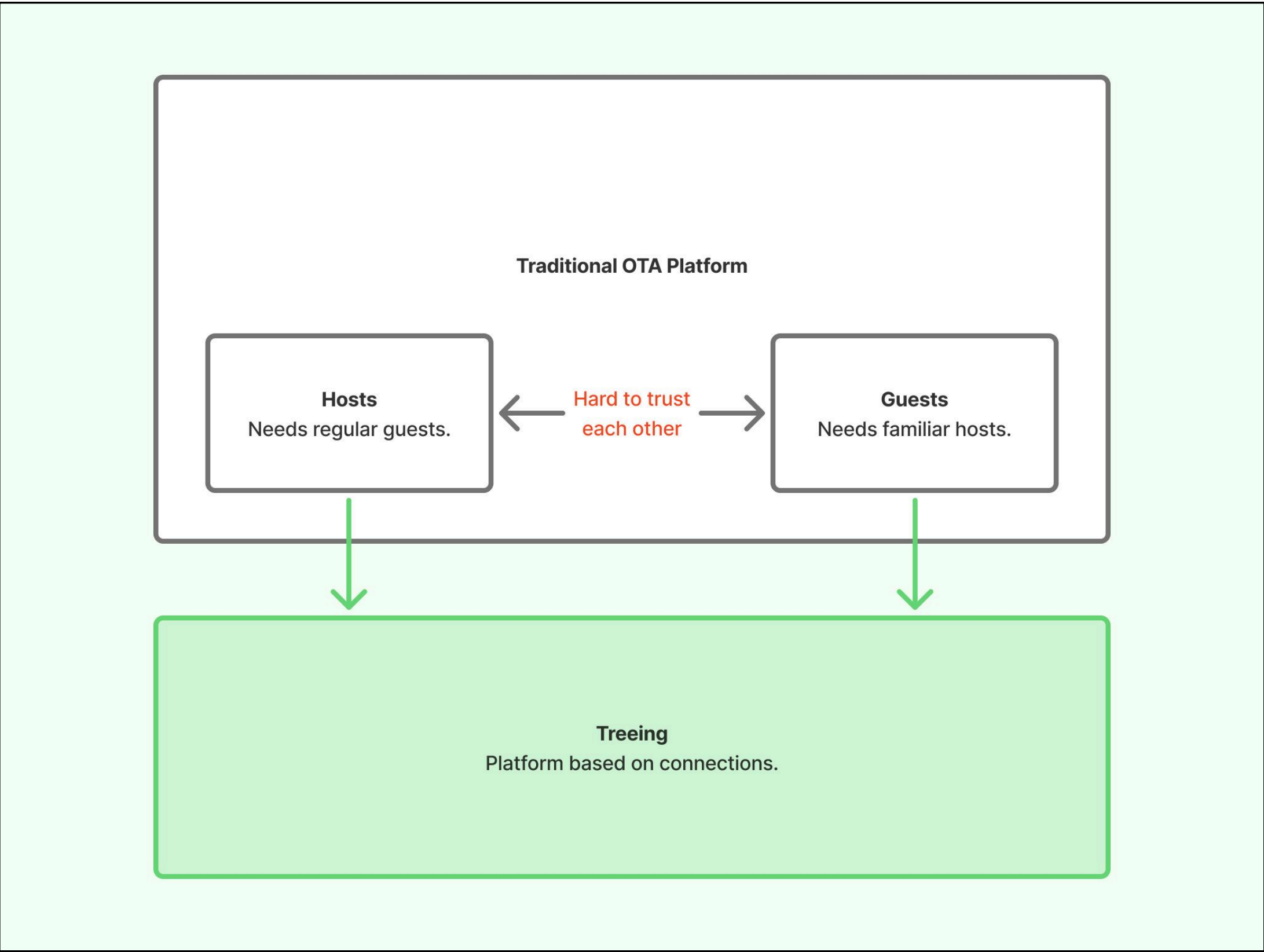
See More

INSTANT CONFIRM

Room Information

Treeing: 为什么旅行预订需要一层关系？

传统平台只关注房源列表，忽略旅客与房东的长期信任关系。



传统旅行预订平台通常从房源库存出发。这套模式适合第一次探索，但不适合已经建立过信任的复购场景。旅客可能记得某次入住体验、某位沟通顺畅的房东，或希望再次预订同一位房东管理的其他房源。

Treeing 的机会是把房东与回访旅客之间已有的信任关系结构化，让旅行预订从一次性搜索交易，转向围绕关系、复购和运营效率展开的产品系统。

Treeing: 把熟客关系变成三方都能受益的结构

旅客、房东和平台的价值来自同一层关系网络。

Treeing 的商业逻辑建立在旅客、房东和平台三方价值之上。旅客降低重复决策成本，房东降低复购经营成本，平台则从房源库存扩展到关系网络。



旅客价值

快速回到可信房东，减少重复搜索。



房东价值

统一管理熟客，提高复购率。



平台价值

关系成为平台资产，提升留存。

旅客搜索复杂、房东维护成本高，是设计优化的核心问题。

在梳理旅客与房东的使用场景时，我把问题分成两类：一类发生在预订前，另一类发生在长期运营中。旅客侧是搜索压力和信任迁移，房东侧是关系管理和维护成本。



旅客搜索重复

信任过的房东很难成为下一次预订入口。



信任关系无法保存

用户记得体验，平台却没有形成可回访路径。



房东熟客分散

常客来自多渠道，缺少统一管理方式。



运营维护成本高

价格、日期和可订状态需要在多处重复调整。

Treeing: 核心假设：让关系成为预订入口

可信关系如果能被发现、管理和预订，就能缩短旅客路径并降低房东运营成本。

如果 Treeing 能把房东与熟客之间的信任关系变成可发现、可管理、可预订的产品结构，那么旅客可以更快完成可信预订，房东也能用更低成本维护复购关系。



关系优先

体验围绕信任关系，而非房源列表。



低采用成本

房东无需放弃已有渠道即可使用。



操作清晰度

房东快速理解状态，完成维护。

V1 聚焦验证熟客关系是否能缩短预订路径，减少旅客搜索压力。

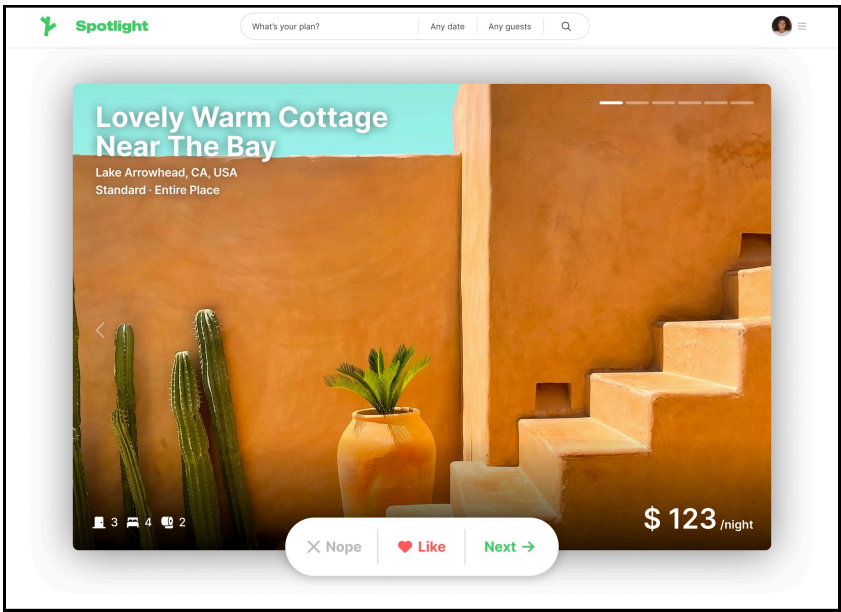
Treeing 的第一版没有试图覆盖完整旅行预订平台，而是先验证一个更具体的问题：当旅客与房东之间已经存在信任关系时，平台能否把这层关系转化为更短、更自然的预订路径？

因此，V1 的设计优先级集中在三件事上：

- 旅客需要一个回到可信房东的入口。
- 房东需要一个管理熟客的基础模型。
- 房东端必须足够轻量。

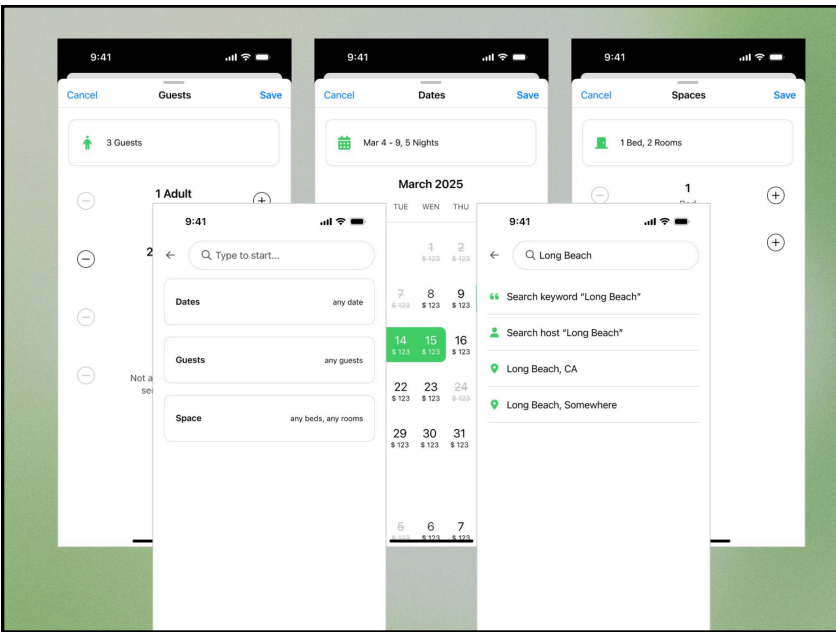
Treeing: 降低旅客的搜索压力

用轻量推荐队列和分步移动搜索减少小屏决策压力。



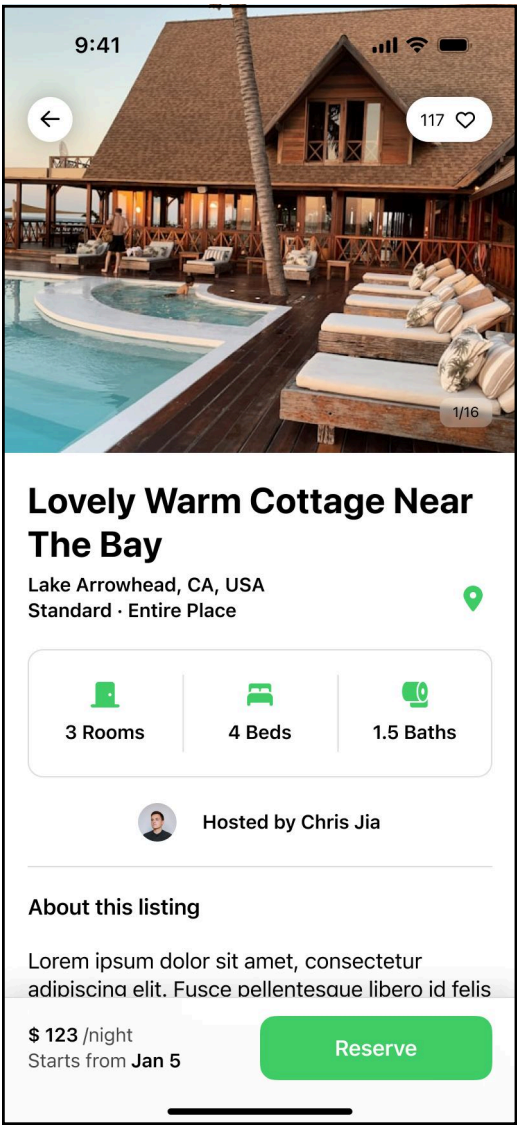
Spotlight

通过推荐队列降低初期筛选压力。



Mobile Search

分步决策：目的地、日期、人数。



Mobile Listing

在小屏幕上保持价格和行动入口清晰。

传统搜索要求用户一开始就明确目的地、日期、预算、房型和偏好，但很多旅行决策并不是这样发生的。Spotlight 被设计成轻量推荐队列，每一次交互既是探索，也是反馈。

移动端搜索采用分步流程，而不是把桌面搜索栏压缩到手机里。目的地、日期和人数被拆到独立页面或模态层中，用户一次只处理一个问题。

Treeing: 把熟客关系产品化：Guest List 与 Invitation

关系先被管理，再被解释成用户愿意加入的价值。

Dashboard Listings Reservations Guest List Billing & Income Marketing Docs Settings				
User	Email	Phone	Status	Actions
 Zoya Patel	zoya.patel@gmail.com	+1 555 123 4567	Invited	 
 Avery Chen	avery.chen@gmail.com	+1 555 987 6543	Guest Listed	 
 Kai Tanaka	kai.tanaka@gmail.com	+1 555 246 8012	Pending	 
 Priya Sharma	priya.sharma@gmail.com	+1 555 789 0123	Blocked	 

Q

Search

Sort by:

Alphabet

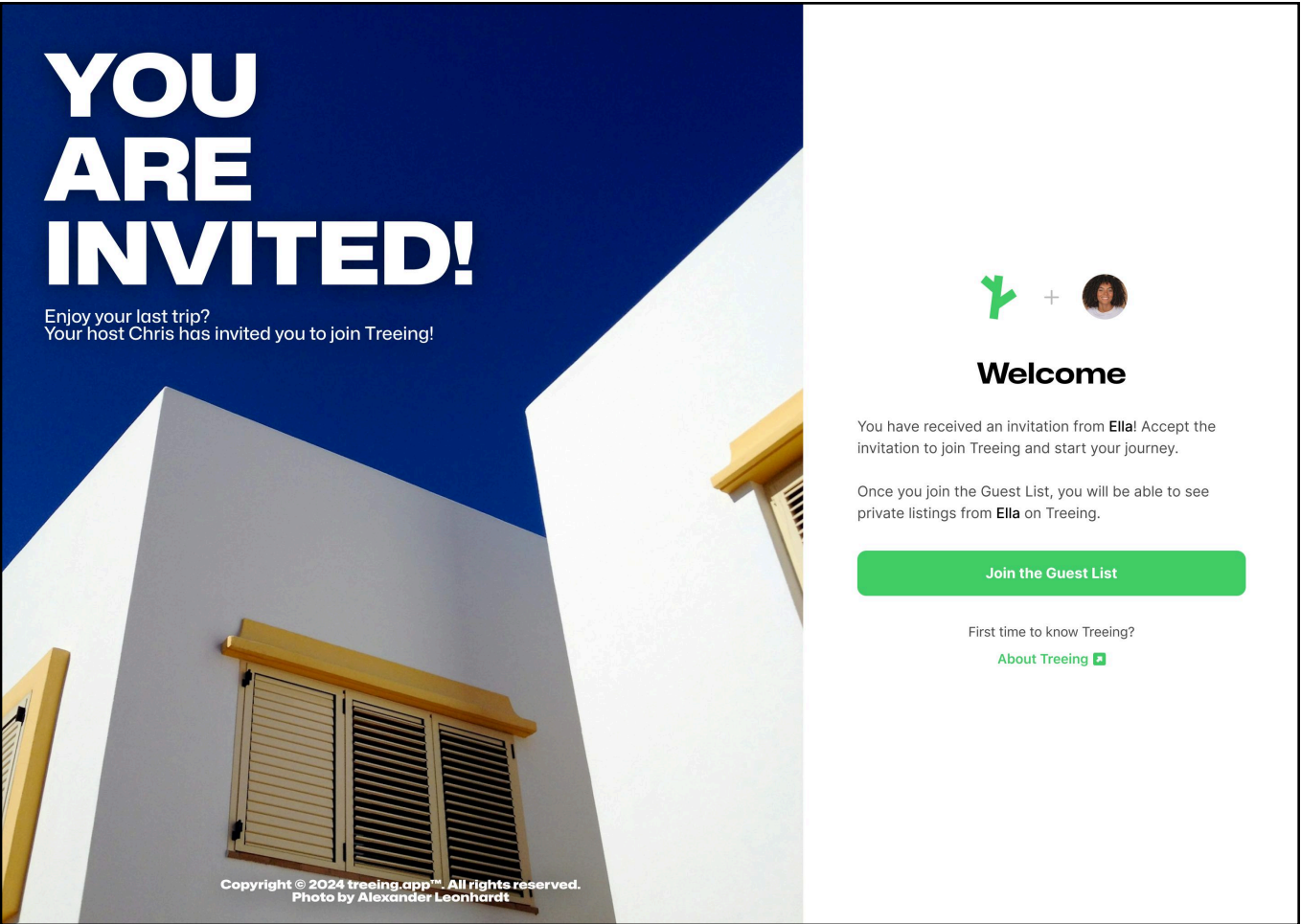
10 Users

Invite by Email

Invite Link

Guest List

把熟客关系转化为可管理客群。



Invitation

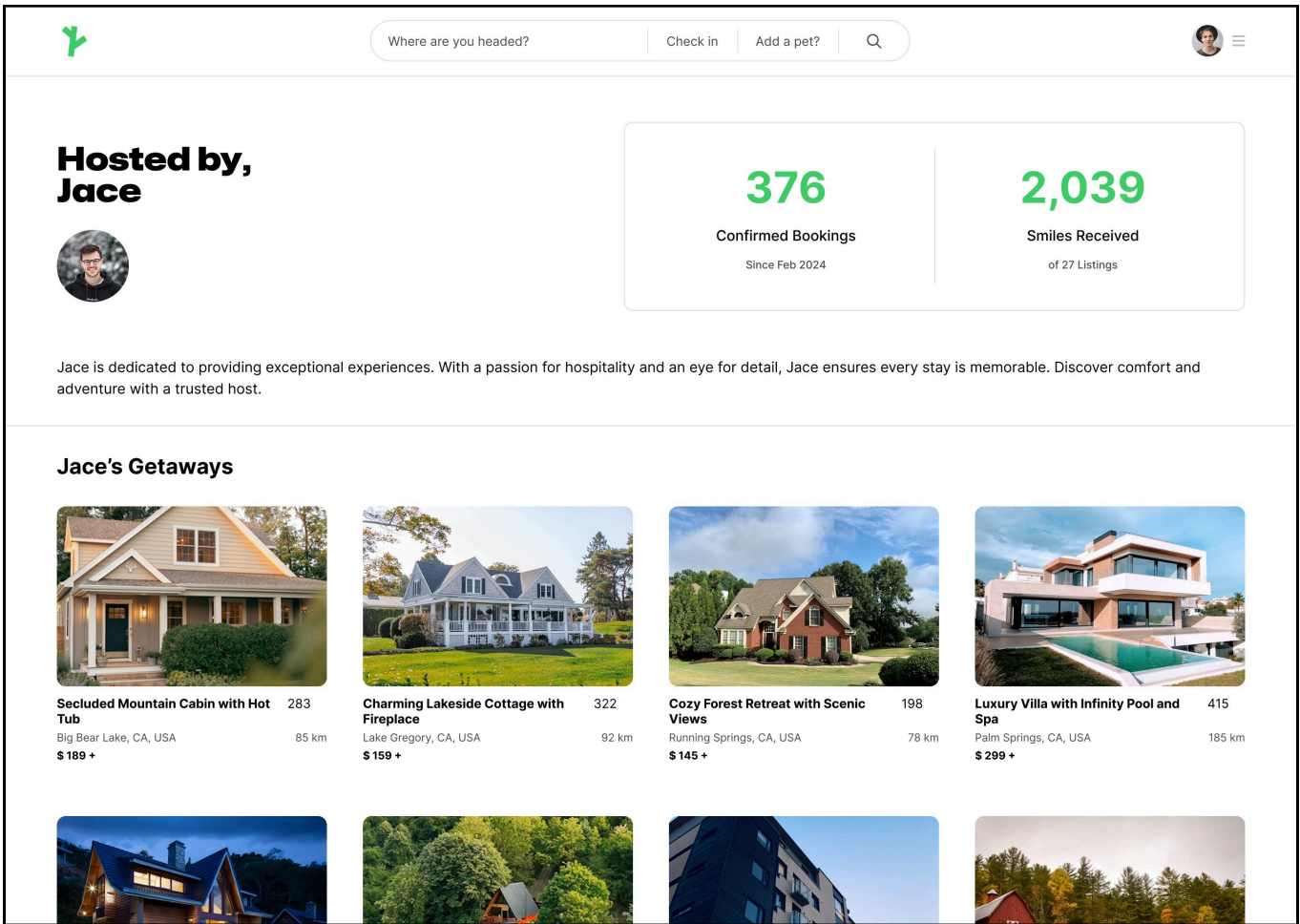
先展示加入价值，再引导操作。

Treeing 的核心不是 Guest List 这个单一功能，而是围绕它建立一整层关系体验。对房东来说，Guest List 是关系管理模型，把不同渠道里的常客整理成可管理客群。

Invitation 流程在这里很关键。用户不是被直接要求加入列表，而是先看到加入 Guest List 的价值：更容易回访可信房东，可能获得更好的价格或专属内容，也减少下一次预订的搜索成本。

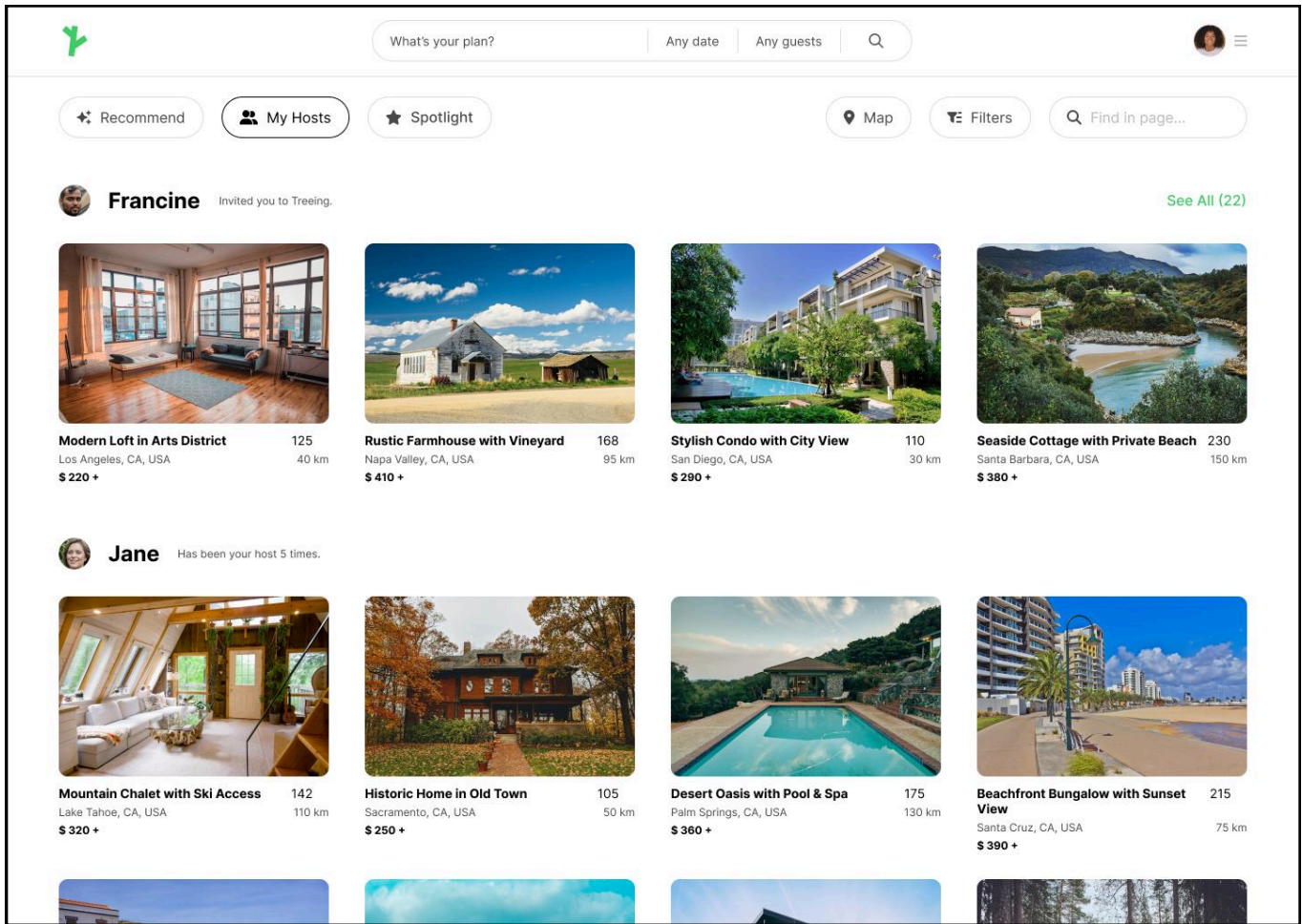
Treeing: 把熟客关系产品化：Host Page 与 My Hosts

关系需要在旅客端变成稳定、可回访的入口。



Host Page

提升房东为可回访目的地。



My Hosts

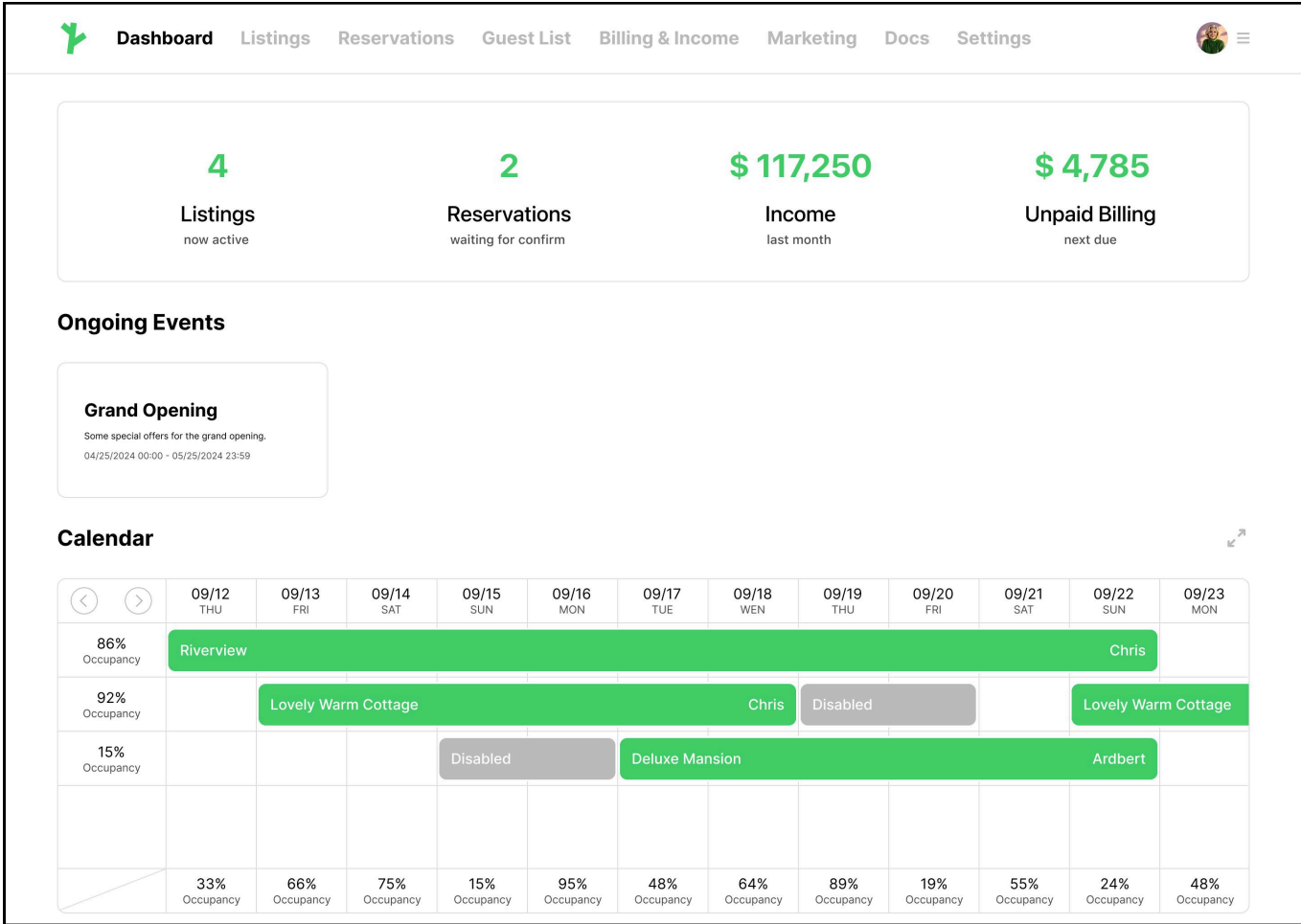
快速访问已信任房东的房源。

对旅客来说，加入 Guest List 后，关系需要变得可见。My Hosts 提供长期入口，让用户快速回到可信房东。Host Page 则把房东从单个房源背后的运营者，提升为可以被关注、理解和再次访问的目的地。

这一层体验让可信房东不再只是某次订单中的联系人，而是可以持续回访和复购的目的地。

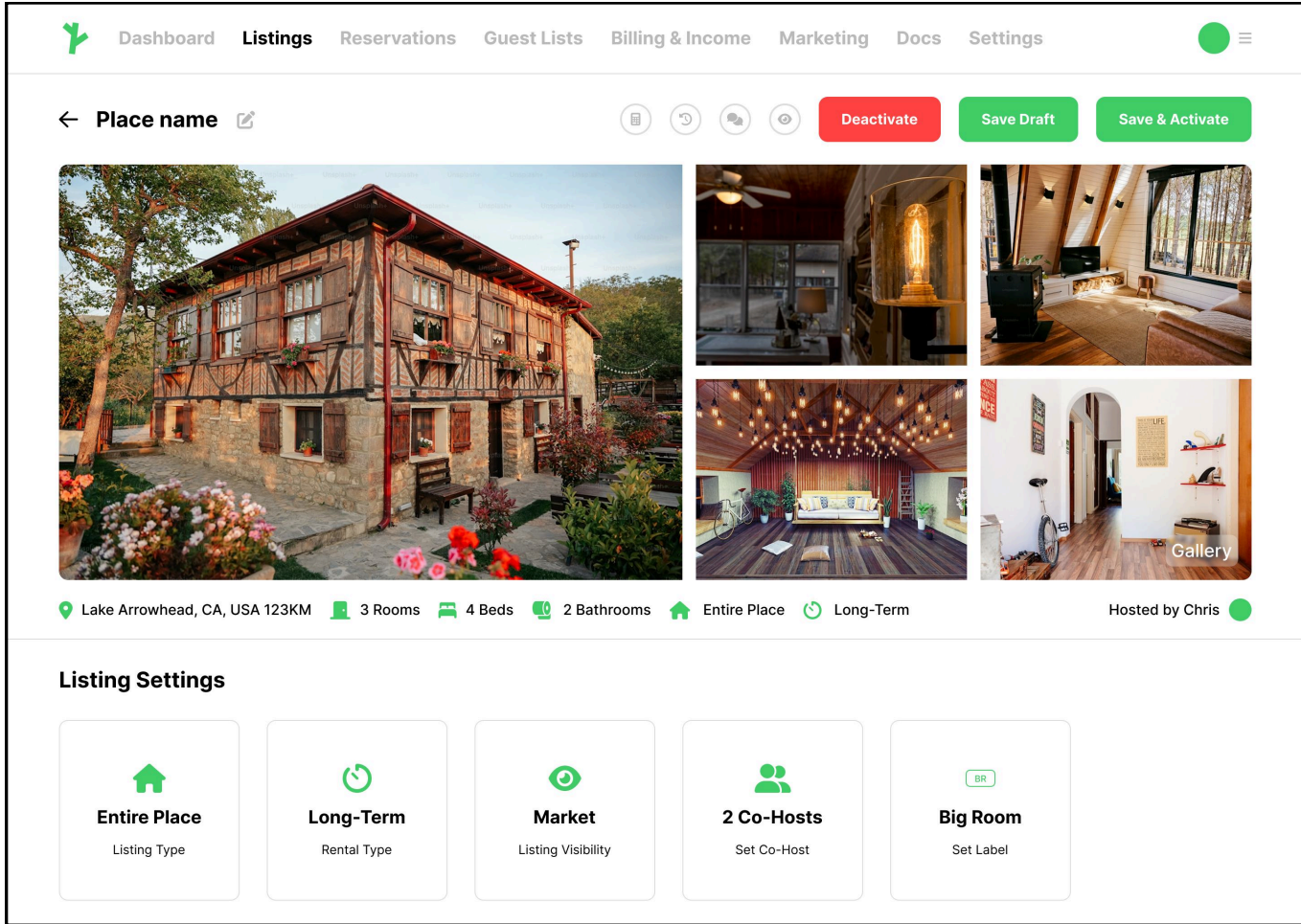
Treeing: 降低房东的运营成本：Dashboard 与 Listing Editing

日常经营信号要一眼可见，编辑方式要接近最终结果。



Dashboard

关键信息一眼可见。



Listing Editing

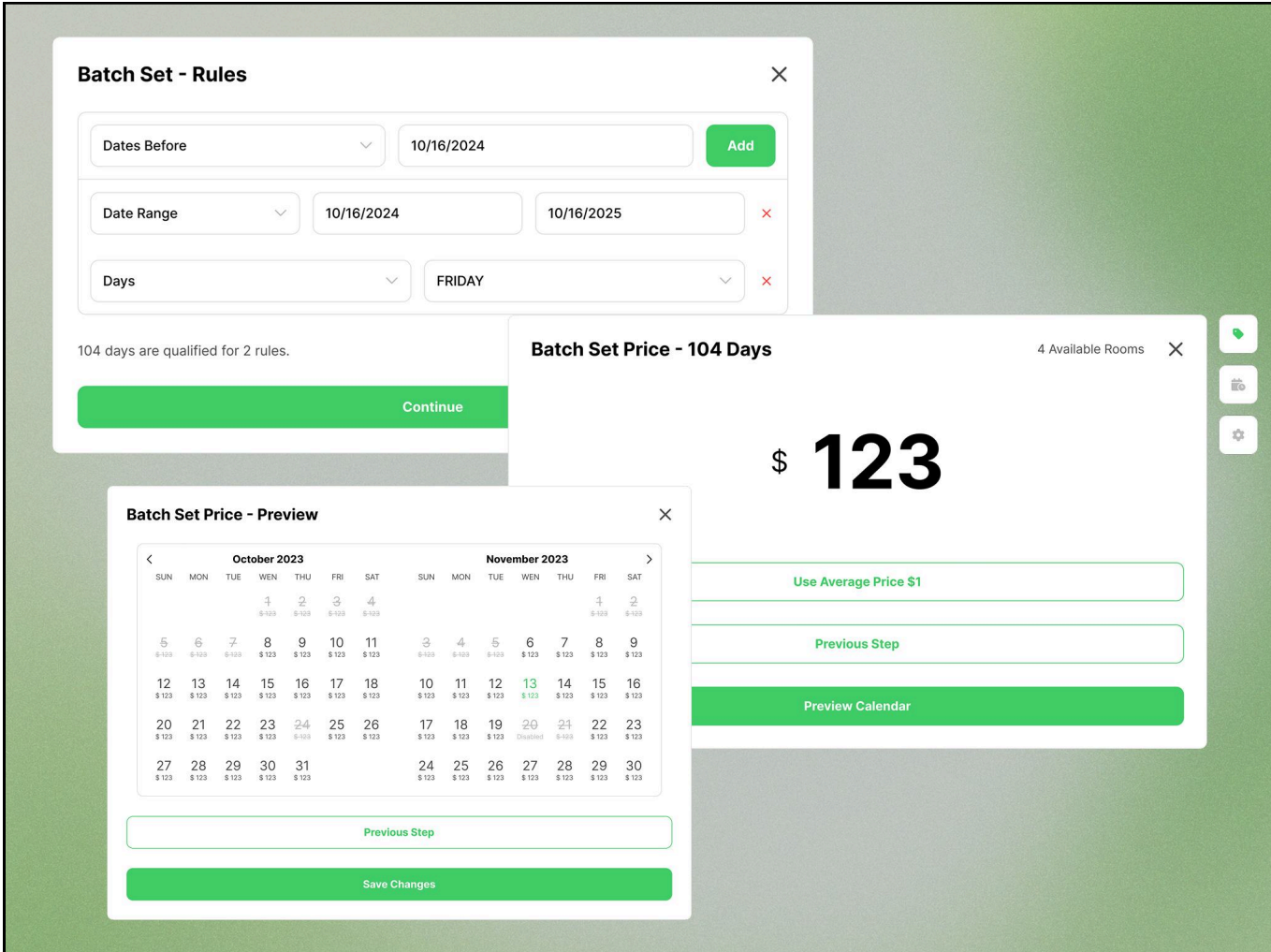
直接在接近最终展示的页面修改房源。

如果 Treeing 只改善旅客端体验，却没有降低房东的维护成本，平台很难长期成立。Host Dashboard 被设计成日常工作台，而不是单纯的数据展示页，关键经营信号、预订状态和日程信息在进入页面时立即可见。

Listing Editing 采用接近旅客视角的直接编辑方式。房东不需要在抽象后台表单中猜测最终展示效果，而是在接近最终页面的结构里修改内容。

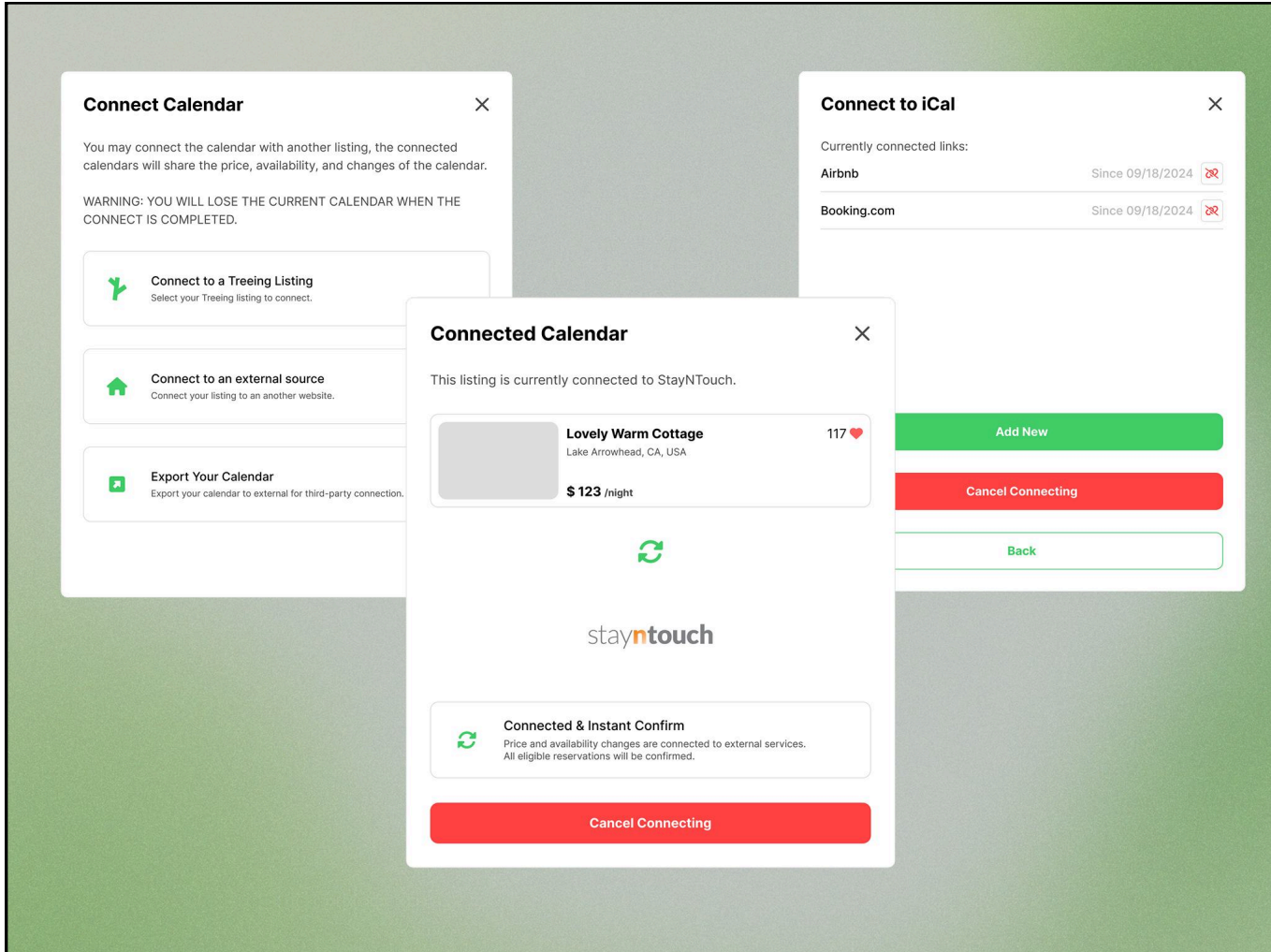
Treeing: 降低房东的运营成本：Batch Set 与 Calendar Connect

批量规则和外部同步降低多房源、多平台维护负担。



Batch Set

批量调整价格和可订状态。



Calendar Connect

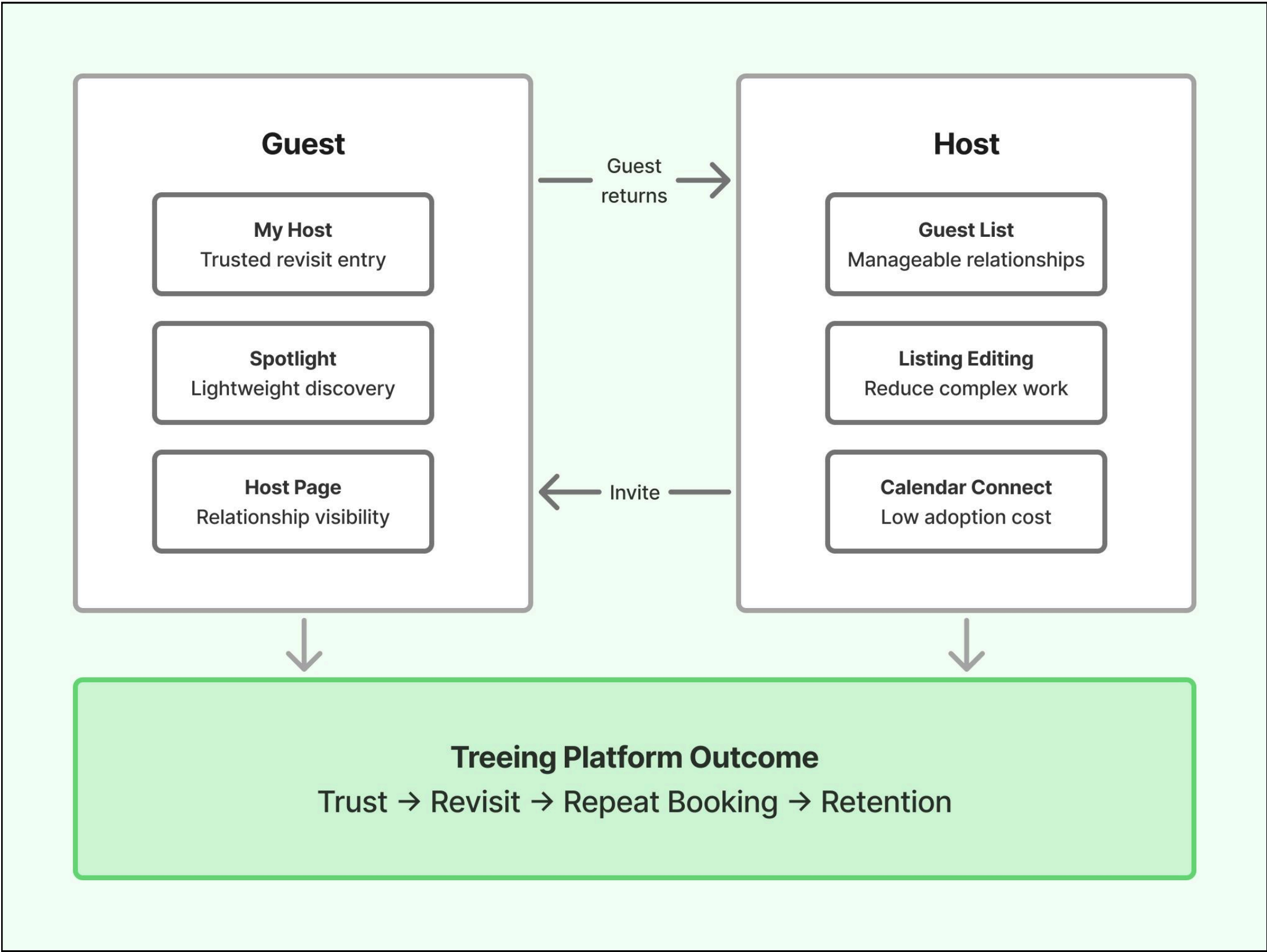
同步外部日历，降低维护成本。

Batch Set 把重复的价格和可订状态调整变成批量规则，减少房东跨多个房源和日期手动修改的风险。Calendar Connect 通过外部平台同步降低接入成本。

Treeing 的目标不是要求房东放弃已有渠道，而是成为更容易接入的复购关系层。

Treeing: 从功能集合到关系型预订系统

功能通过关系链连接，形成可复购、可持续运营的系统。



Treeing 的最终设计不是一组分散功能，而是一套围绕关系运转的产品系统。旅客端负责发现和回访，Spotlight 降低首次探索压力，My Hosts 和 Host Page 让可信房东成为可回访入口。

房东端负责关系管理和运营维护。Guest List 管理熟客关系，Dashboard 提供日常经营视图，Listing Editing 降低编辑理解成本，Batch Set 和 Calendar Connect 处理多房源、多平台维护负担。

设计收益可以从旅客、房东和平台三个层面理解。

Treeing 的设计收益不只是单页优化，而是让旅客更容易回到可信房东，让房东更容易维护熟客关系，让平台从一次性交易转向长期复购。



旅客

减少重复搜索成本，提供复购渠道。



房东

降低多房源、多平台维护成本，提供管理熟客的平台。



平台

从一次性交易转向关系驱动的复购。

项目反思：设计重点是关系如何成为预订入口，而非单页优化。

Treeing 让我重新理解旅行预订产品的设计重点。这个项目一开始看起来像一个房源平台，但真正复杂的地方不是展示更多房源，而是判断平台应该围绕什么建立长期价值。

如果只从旅客端看，问题会被理解成搜索体验不够好。如果只从房东端看，问题又会变成后台工具不够高效。但 Treeing 的关键在于，这两个问题其实是同一个系统的两端。

